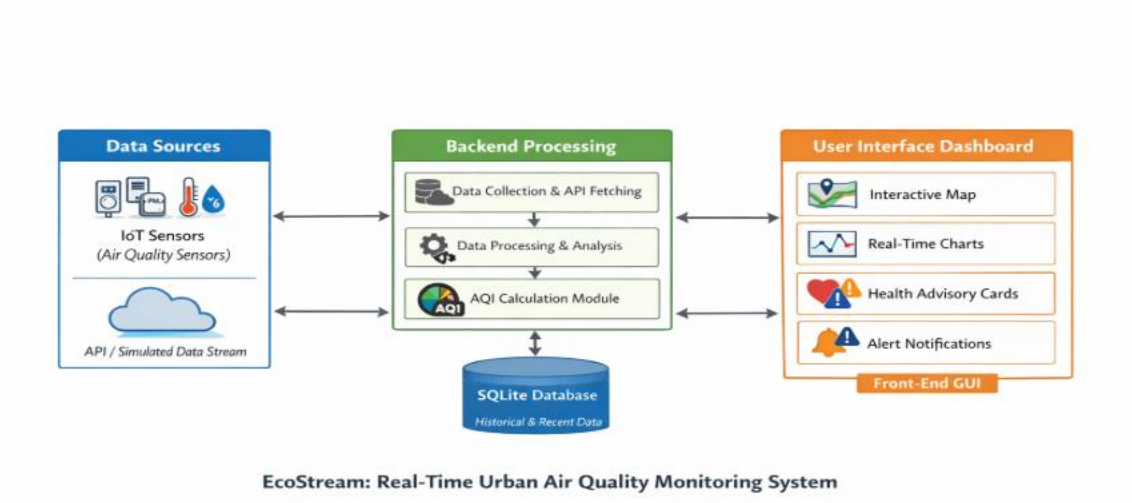
Milestone 2: Design Document

Project: EcoStream: Real-Time Urban Air Quality Monitoring & Analytics Dashboard

1. **ARCHITECTURE DIAGRAM**



2. MODULE BREAKDOWN

The major files and modules that involved in these are:

1. main.py – This is the main file that starts the application. It connects all modules and launches the dashboard.

2. data_fetcher.py – This module collects air quality data from APIs or simulated sensors at regular intervals.

3. data_processor.py – This module cleans and formats the collected data by removing errors and preparing it for analysis.

4. aqi_calculator.py – This module calculates the Air Quality Index (AQI) using pollutant concentration values.

5. database_manager.py – This module stores the processed data in an SQLite database and retrieves historical data when needed.

6. dashboard.py – This module creates the interactive dashboard with graphs, maps, and alerts to display the air quality information.

These files work together to **collect data → process it → calculate AQI → store it → visualize it for users.**

3. DATA FLOW EXPLANATION

EcoStream starts by collecting environmental data from sensors or external APIs at regular intervals of **30–60 seconds**. These sensors gather information such as air quality levels and other environmental conditions. After the data is collected, it goes through a **data validation and cleaning process** where missing, incorrect, or inconsistent values are removed. The system also standardizes timestamps and ensures that all measurement units are consistent.

Next, the system performs **Air Quality Index (AQI) calculation and threshold analysis**. It calculates AQI scores based on the environmental data and assigns health categories that indicate whether the air quality is good, moderate, or hazardous. The system also checks if the AQI values exceed predefined safety thresholds.

Once the data is processed, it is stored in a **database** for both real-time monitoring and future analysis. The stored data allows the system to maintain historical records and track long-term environmental trends. Finally, the processed information is displayed on a **dashboard**, which shows heatmaps, real-time charts, KPI cards, and alert notifications to help users understand current environmental conditions.

4.External Tools / Libraries Justification

Pandas – Used to clean, organize, and analyze air quality data collected from sensors or APIs.

NumPy – Used for performing numerical calculations such as computing Air Quality Index (AQI) values.

Flask – Used as the backend framework to run the application and handle communication between modules.

Plotly – Used to create interactive charts and graphs to visualize pollution trends.

SQLite – Used to store and retrieve air quality data for historical comparison and analysis.

5. Risk Assessment (Technical Challenges)

1. Real-time performance lag
Frequent data updates may slow down the dashboard when multiple visualizations refresh at

the same time. To handle this, the system will use optimized data processing and update only the most recent data instead of reloading the entire dataset.

2. Data parsing or missing values
Data received from APIs may contain missing or incorrect values. This will be handled by implementing data validation and cleaning steps before processing the data.

3. API or data source failure
Sometimes the external API may become unavailable or return incomplete data. To reduce this risk, the system will include error handling and a fallback simulated data source.

4. Database storage errors
Continuous data storage may lead to database conflicts or file errors. Proper exception handling and regular data checks will be used to ensure reliable storage and retrieval.

6. Research References

1. U.S. Environmental Protection Agency (EPA). (2023). *Air Quality Index (AQI).*
https://www.epa.gov/air-quality-index
This source explains how the Air Quality Index is calculated and how air pollution levels are categorized. It helps ensure the project uses accurate and standardized AQI calculations.

2. McKinney, W. (2010). *Data Structures for Statistical Computing in Python.*
https://doi.org/10.25080/Majora-92bf1922-00a
This paper introduces the Pandas library used for data analysis in Python. It supports the use of Pandas for cleaning and processing environmental data in the project.

3. Plotly Technologies Inc. (2024). *Plotly Python Documentation.* https://plotly.com/python/
Plotly documentation explains how to create interactive charts and dashboards. It is used in the project to visualize air quality data using graphs and maps.